Progress Note: Synthetic CPU/GPU Ambiguity-Resolution Benchmarking

6344837 - Sorin Bețișor

Overview

This note summarizes the current post-fix benchmarking status of the `traccc` greedy ambiguity resolver on synthetic datasets. The focus here is on two questions:

- whether the patched GPU resolver is now **correct**, and
- whether it is **faster than the CPU baseline** on the current synthetic benchmark family.

Synthetic dataset setup (using acts/traccc pipeline)

- a fixed random seed of 42, with `warmup=3` and `repeats=10`;
- candidate-count tiers $n \in \{1000, 5000, 10000\}$;
- three synthetic ambiguity-density presets labelled `low`, `med`, and `high`;

These datasets are useful for controlled stress-testing because they let candidate count and ambiguity density be varied systematically. However, they are still synthetic inputs, so they should be treated as an intermediate benchmark stage rather than a physics-representative conclusion.

Main findings

- Correctness: all nine post-fix configurations report a hash match between the CPU and GPU versions.
- GPU resolver-only time (kernel-side benchmark time) is better than CPU in 5/9 synthetic cases, essentially at parity in `n5000_med`, and worse for all three $n = 1000$ cases.
- End-to-end GPU time including host-to-device and device-to-host transfers is better than CPU in only 2/9 cases: `n10000_low` and `n10000_med`. The `n10000_high` case is almost break-even.
- This means the current synthetic sweep already shows a kernel-level GPU advantage at larger sizes, but also shows that data transfer dominates for small and medium inputs.
- The broader upstream test result at 50,000 tracks is much more encouraging: the patched GPU resolver is reported as about 8-13× faster than CPU including transfers.

Compact result table

Config	CPU ms	GPU ms	GPU e2e ms	CPU/GPU	CPU/GPU e2e
n1000_low	3.58	10.51	16.19	0.34	0.22
n1000_med	4.23	18.15	22.94	0.23	0.18
n1000_high	3.46	8.14	12.64	0.43	0.27
n5000_low	40.54	26.73	48.59	1.52	0.83
n5000_med	34.47	34.66	55.25	0.99	0.62
n5000_high	22.66	13.34	34.32	1.70	0.66
n10000_low	142.33	34.24	75.16	4.16	1.89
n10000_med	80.27	36.06	76.56	2.23	1.05
n10000_high	59.61	21.01	61.45	2.84	0.97

Table 1: Post-fix synthetic benchmark summary. "GPU ms" is resolver-only time; "GPU e2e ms" includes transfer overhead. Ratios above 1 indicate a GPU advantage relative to CPU.

Interpretation

If the question is "is the GPU kernel faster than the CPU resolver?", then the answer is **yes for the larger synthetic workloads**, especially at $n = 10{,}000$, where the resolver-only speedup ranges from about 2.2× to 4.2×.

Figures

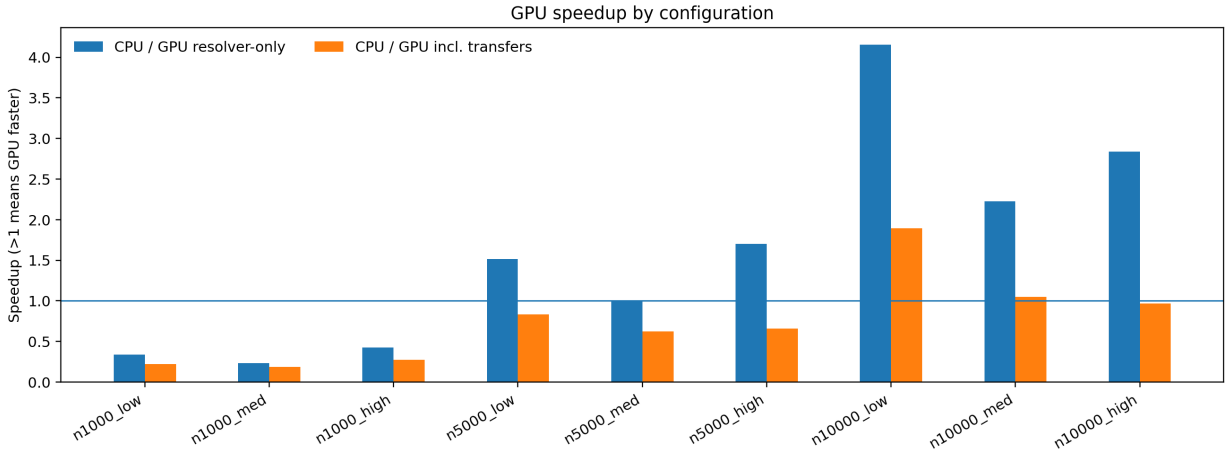

Figure 1: Resolver-only timing comparison between CPU and GPU across the synthetic benchmark configurations.

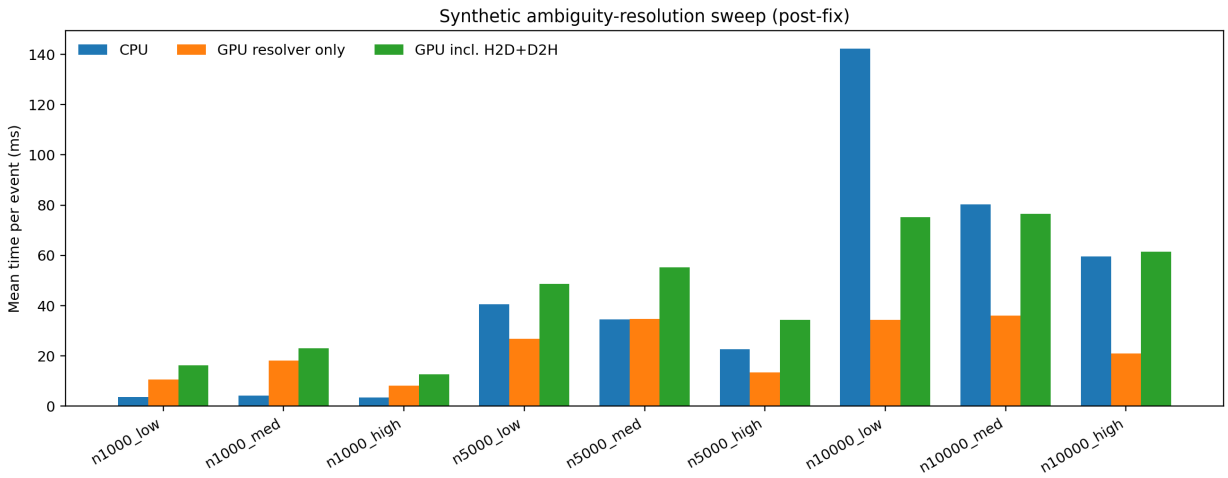

Figure 2: End-to-end timing comparison including host-to-device and device-to-host transfer overhead.

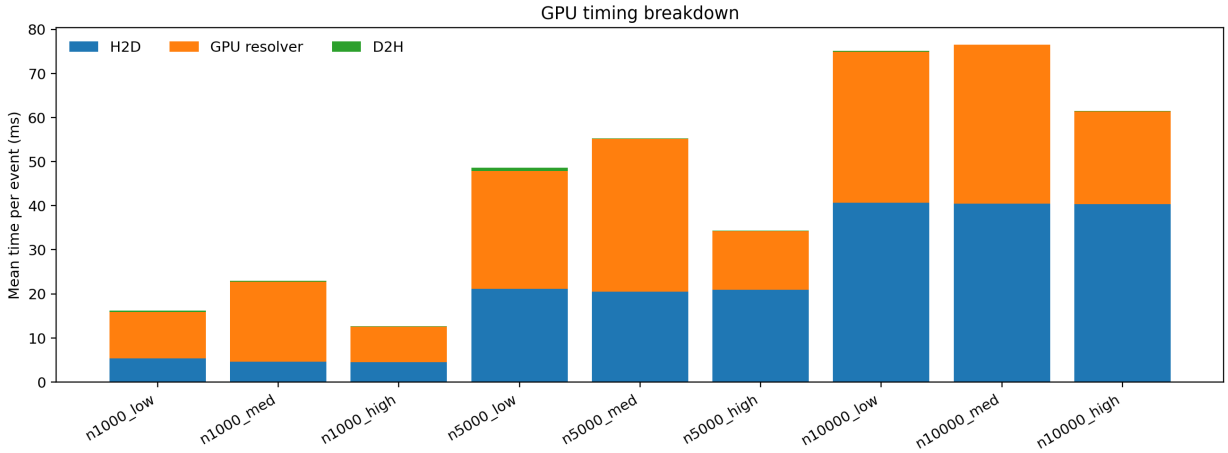

Figure 3: GPU Timing Breakdown - H2D,D2H, Solver.

Discussion

The synthetic sweep now supports two strong interim conclusions.

First, the bug fix was necessary and successful from a correctness perspective: the GPU and CPU now agree exactly on all nine tested synthetic configurations (and on the disabled tests from the traccc codebase). This is important because performance claims are only meaningful once the selected track set is verified to match the reference implementation.

Second, the GPU already shows a clear compute-side advantage on larger workloads. In particular, the $n = 10{,}000$ cases show that the patched resolver kernel is substantially faster than the CPU baseline. However, these gains are only partially visible in the current end-to-end path because transfer costs remain a large fraction of total runtime.

This suggests that the GPU algorithm itself is promising, but that a fair final evaluation should be performed on workloads where transfer overhead is better amortized or where more of the pipeline remains resident on the device.